

TECHNICAL REPORT

TCA ERP

Enterprise Resource Planning for a visa consulting agency

Web application (React + FastAPI + PostgreSQL), local AI, and native iOS app

Client	Tunisie Conseil et Assistance (TCA) — Tunis, Tunisia
Author	Wisseem Sahli — Full-Stack Developer
Document date	02 October 2026
Scope	Architecture, data model, UML, REST API, algorithms, security, deployment

Contents

1. Executive summary	3
2. Business context and functional scope	3
2.1 Functional modules	3
2.2 UML use-case diagram	4
2.3 Non-functional requirements	4
3. Technology stack and rationale	5
4. System architecture	6
4.1 Request lifecycle	6
4.2 Repository layout	7
5. Database design	7
5.1 Tables	8
5.2 Design decisions worth explaining	9
6. Backend	9
6.1 Application structure	9
6.2 Authentication and authorization	9
6.3 REST API reference	10
6.4 Passport extraction pipeline	11
6.5 AI assistant with live business context	11
6.6 Payroll engine (fiche de paie)	12
6.7 Other business rules	13
7. Web frontend (React)	14
7.1 Structure	14
7.2 State management	14
7.3 Internationalisation, theming and responsiveness	14
8. Native iOS application (SwiftUI)	15
9. Security review	16
9.1 Known limitations and planned hardening	16
10. Deployment and operations	16
11. Challenges solved	17
12. Interview preparation: likely questions	17
Appendix A — Configuration variables	18
Appendix B — Glossary	19

1. Executive summary

TCA ERP is a full-stack business application built for **Tunisie Conseil et Assistance (TCA)**, a visa and travel consulting agency operating in Tunisia and Libya. It replaces spreadsheets and paper with one system covering the agency's daily work: client and visa case management, accounting (invoices, receipts, treasury, bank accounts), human resources (employee requests and payslips), analytics, and an AI assistant that runs entirely on the agency's own hardware.

The system has **one backend and two clients**: a **FastAPI** REST API over **PostgreSQL**, a **React** single-page web application used by up to eight staff at once, and a **native SwiftUI iPhone app** that lets the agency owner follow the business from anywhere. Both clients use the same endpoints and the same JSON Web Token (JWT) authentication, so business rules live in one place.

Key facts

Aspect	Summary
Users & roles	Two roles: Admin (owner, accounting, payroll, user management) and Agent (clients, requests, stats). Sign-up requires admin approval by e-mail.
Backend	Python 3, FastAPI, SQLAlchemy (SQLAlchemy + Pydantic), Alembic migrations, ~3,800 lines across 13 modules, 54 REST endpoints.
Database	PostgreSQL, 12 tables, all persistent state in one database; uploaded files on disk with paths stored in the DB.
Web client	React 19 + Vite 8, React Router 7, Zustand state, plain CSS with design tokens, bilingual English/Arabic with full right-to-left layout, light/dark themes, responsive.
iOS client	SwiftUI (iOS 17+), ~6,100 lines, async/await networking, Keychain token storage, EN/AR.
AI (local)	Ollama serving the Qwen2.5-VL 3B vision-language model for the chat assistant and passport reading fallback; a deterministic OCR + ICAO-9303 MRZ pipeline reads most passports in ~2 s.
Documents	Invoices, receipts and Arabic payslips generated as PDFs with LaTeX (XeLaTeX + Amiri font).
Payroll	Imports the ZKTeco K40 time-clock export (.xls/.xlsx) and computes base pay, +25% overtime, +50% weekend and +100% holiday hours, advances and net pay.

2. Business context and functional scope

TCA prepares visa files for clients (passport data, documents, appointments), sells related services (flights, hotel reservations), invoices them, and runs a small team. Before the ERP, client data, payments and staff hours were tracked in separate Excel files, which made follow-up ("whose passport is ready?", "how much did we spend this month in Libya?") slow and error-prone.

2.1 Functional modules

Module	What it does	Who
Authentication	Sign-in, account requests with e-mail approval, role management, "remember me" sessions.	All / Admin
Dashboard	Monthly snapshot: treasury net, bank reserves, client counts by visa status, alerts.	All
Clients	Full client file: passport data (auto-extracted from a photo), contact, category (normal / fair), visa status & type, flight and hotel reservation, price and payment state, photo and documents. Search, tabs (all, fair, reservation, alert) and paging.	All
Passport extraction	Upload a passport photo; fields are filled automatically and flagged verified / likely / review.	All

Module	What it does	Who
Accounting	Per country (Tunisia TND, Libya LYD): invoices (factures with VAT and stamp duty) and receipts (reçus) as PDF, treasury ledger (spending / gathering), bank accounts and transactions, monthly stats.	Admin
Employee requests	Vacations, salary advances, loans: every employee submits; each agent sees only their own; the admin sees all and approves or rejects.	All / Admin
Agency requests	Requests the admin addresses to staff, read-only for agents.	Admin
Payroll (fiche de paie)	Time-clock import, pay computation, Arabic payslip PDF, history.	Admin
Stats	Built-in charts plus a chart builder : pick a dataset, a grouping, a metric and a period; charts are saved per user.	All (money: Admin)
AI assistant	Chat that answers from live business data (role-filtered), with optional screenshot input.	All
Users	Pending sign-ups, approve / reject, change role, delete.	Admin

2.2 UML use-case diagram

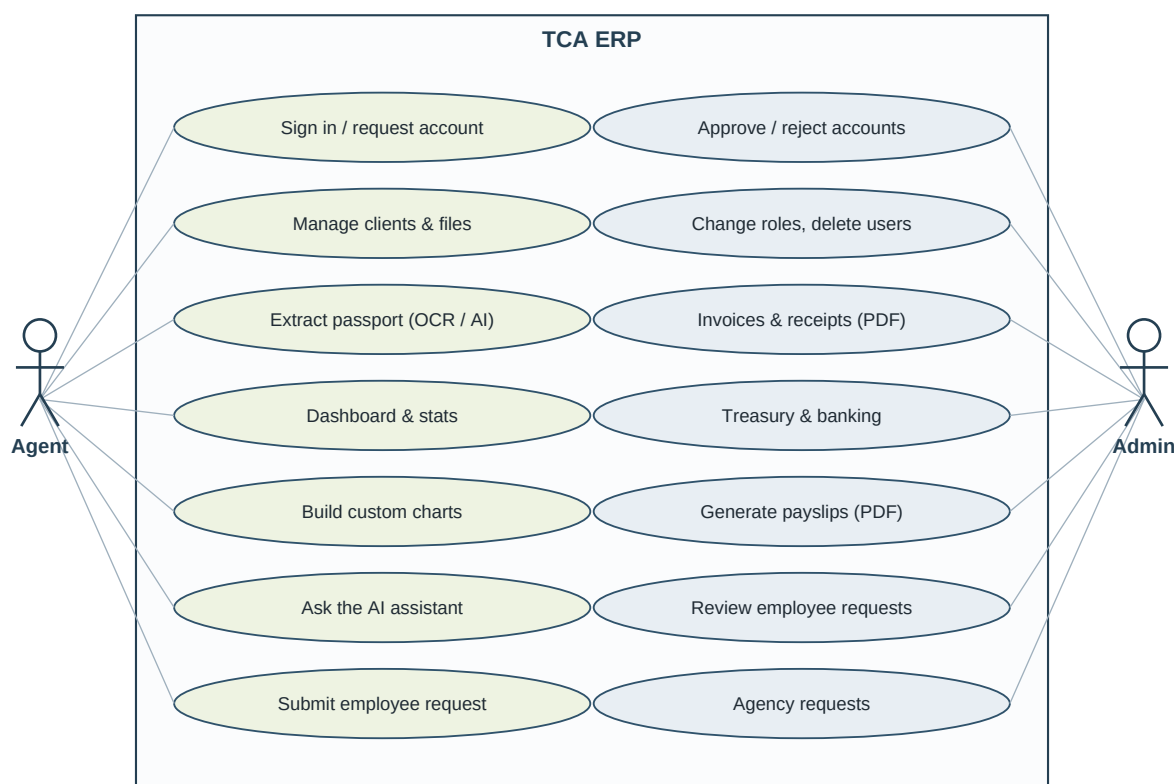


Figure 1 — Use cases by actor. The Admin generalises the Agent.

2.3 Non-functional requirements

- **Data stays on-premises:** client passports and finances never leave the agency's server; the AI model runs locally (no cloud API keys, no per-request cost).
- **Bilingual:** every screen in English and Arabic, with correct right-to-left mirroring; the payslip itself is in Arabic, as required for Tunisian employees.
- **Small concurrency, fast response:** ~8 web users and 2-3 mobile users; interactive pages answer in well under a second, passport OCR in about 2 s.

- **Security:** hashed passwords, signed expiring tokens, server-side role checks on every request, admin approval for new accounts.
- **Maintainability:** one source of truth for rules (computed server-side so web and iOS agree), typed models, versioned database migrations.

3. Technology stack and rationale

Layer	Technology	Why this choice
API	FastAPI + uvicorn (ASGI)	Type-hinted endpoints, automatic validation and OpenAPI docs, async support for slow calls (LLM, OCR) via a thread pool.
ORM / models	SQLModel	One class is both the Pydantic schema (validation, JSON) and the SQLAlchemy table, which removes duplicated model definitions.
Database	PostgreSQL 16 + psycopg 3	Relational integrity (foreign keys, unique passport numbers), date functions for monthly stats, mature and free.
Migrations	Alembic	Every schema change is a versioned, reversible script applied with one command.
Auth	bcrypt + PyJWT (HS256)	Industry-standard password hashing; stateless tokens usable by both clients.
Spreadsheets	openpyxl, xlrd	openpyxl for .xlsx; xlrd for the legacy raw-BIFF .xls written by the ZKTeco software.
PDF	LaTeX (TinyTeX): XeLaTeX, pdfLaTeX	Typographic quality for invoices; XeLaTeX + babel + Amiri renders Arabic right-to-left text.
OCR	RapidOCR (ONNX runtime), OpenCV, optional Tesseract	Free, CPU-only, ~1 s per page; combined with MRZ check digits for certainty.
LLM	Ollama + Qwen2.5-VL 3B	Small vision-language model that fits a consumer GPU; OpenAI-compatible API.
Web UI	React 19, Vite 8, React Router 7	Component model, fast dev server with an /api proxy, client-side routing.
State	Zustand	Minimal global stores (auth, language, theme, UI) without Redux boilerplate.
Charts / motion	Recharts, GSAP, Lenis	Declarative charts; polished animations and smooth scrolling.
iOS	SwiftUI, URLSession, Keychain	Native performance and look; no third-party dependencies.

4. System architecture

The architecture is a classic **three-tier** design: presentation (web and iOS clients), application (FastAPI) and data (PostgreSQL, files), plus two local compute services (OCR/LaTeX in-process, and the Ollama model server). Clients never touch the database or the model directly; everything goes through the authenticated REST API.

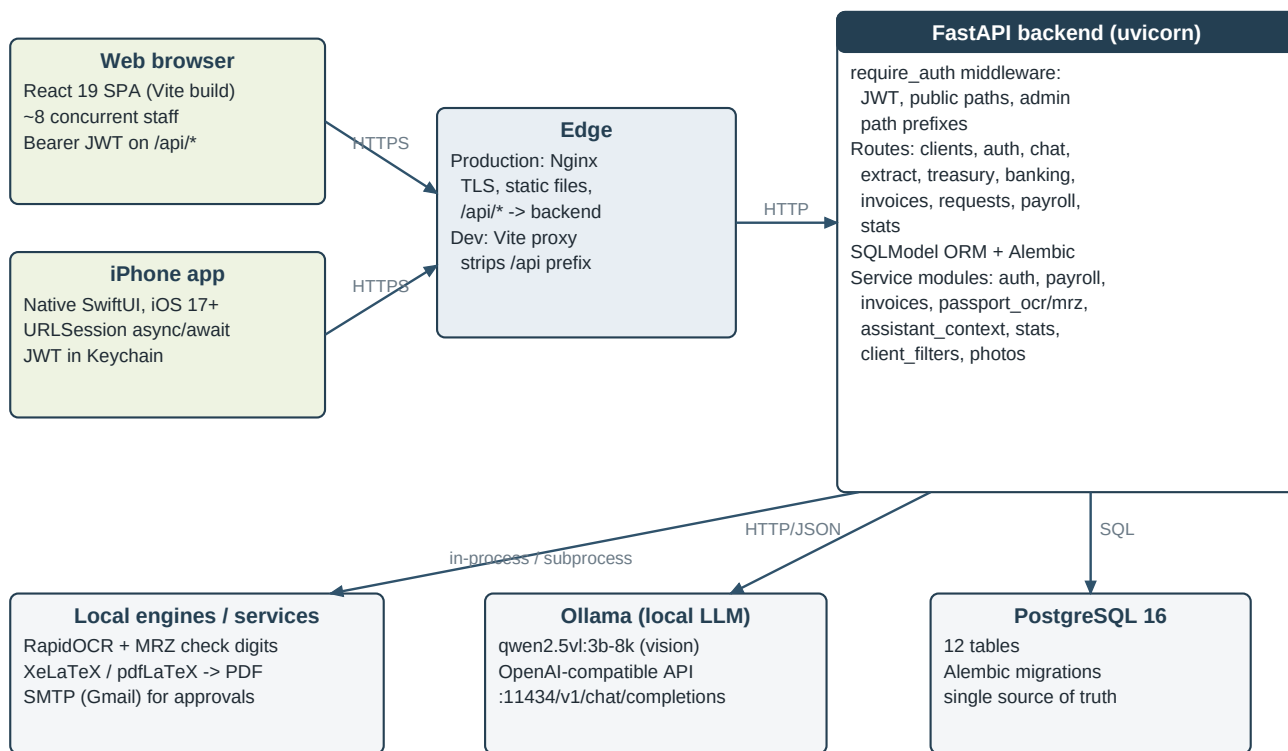
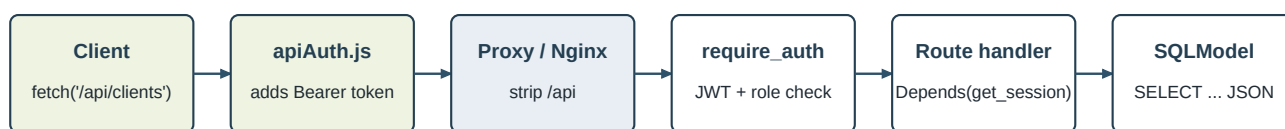


Figure 2 — Deployment / component view.

4.1 Request lifecycle



401 -> the fetch patch logs the user out; 403 -> 'Admin access required'

Figure 3 — Path of an authenticated web request.

The web app always calls relative URLs such as `/api/clients`. In development the Vite dev server proxies `/api` to `localhost:8001` and strips the prefix; in production Nginx does the same. A small module (`src/lib/apiAuth.js`) wraps `window.fetch` once at start-up: for every `/api/` call it adds `Authorization: Bearer <token>` and, on a 401 response, logs the user out. This avoids threading the token through dozens of call sites.

4.2 Repository layout

```

zouhour/
  backend/
    backend.py          FastAPI app: middleware + all routes
    models.py           SQLAlchemy tables and request schemas
    db.py               engine + per-request Session dependency
    auth.py             bcrypt, JWT, current-user / require_admin, approval e-mail
    client_filters.py   search, tabs, alert rules (SQL expressions)
    passport_ocr.py     OCR pipeline
    passport_mrz.py     ICAO 9303 parser
    assistant_context.py live data snapshot for the AI assistant
    stats.py           chart-builder query engine
    invoices.py        LaTeX invoices/receipts
    payroll.py         time clock + payslips
    photos.py          file storage helpers
    seed_admin.py      create the admin
    migrations/        Alembic versions
    tests/             MRZ unit tests
  src/                React app (app/, components/, features/, store/, lib/, i18n/, styles/)
  ios_swift/TCAERP/   SwiftUI app (App/, Core/, Features/, Components/)
  uploads/            runtime files (git-ignored)
  .env               secrets and configuration (never committed)

```

5. Database design

All persistent state lives in one PostgreSQL database. Tables are declared as SQLAlchemy classes in `backend/models.py`; each table class usually extends a *Base* class that holds the shared fields, and a *Create* class is used as the request body, so the same field definitions drive validation, the API schema and the table. Files (photos, scans, PDFs) are stored in `uploads/` and only their relative path is stored.

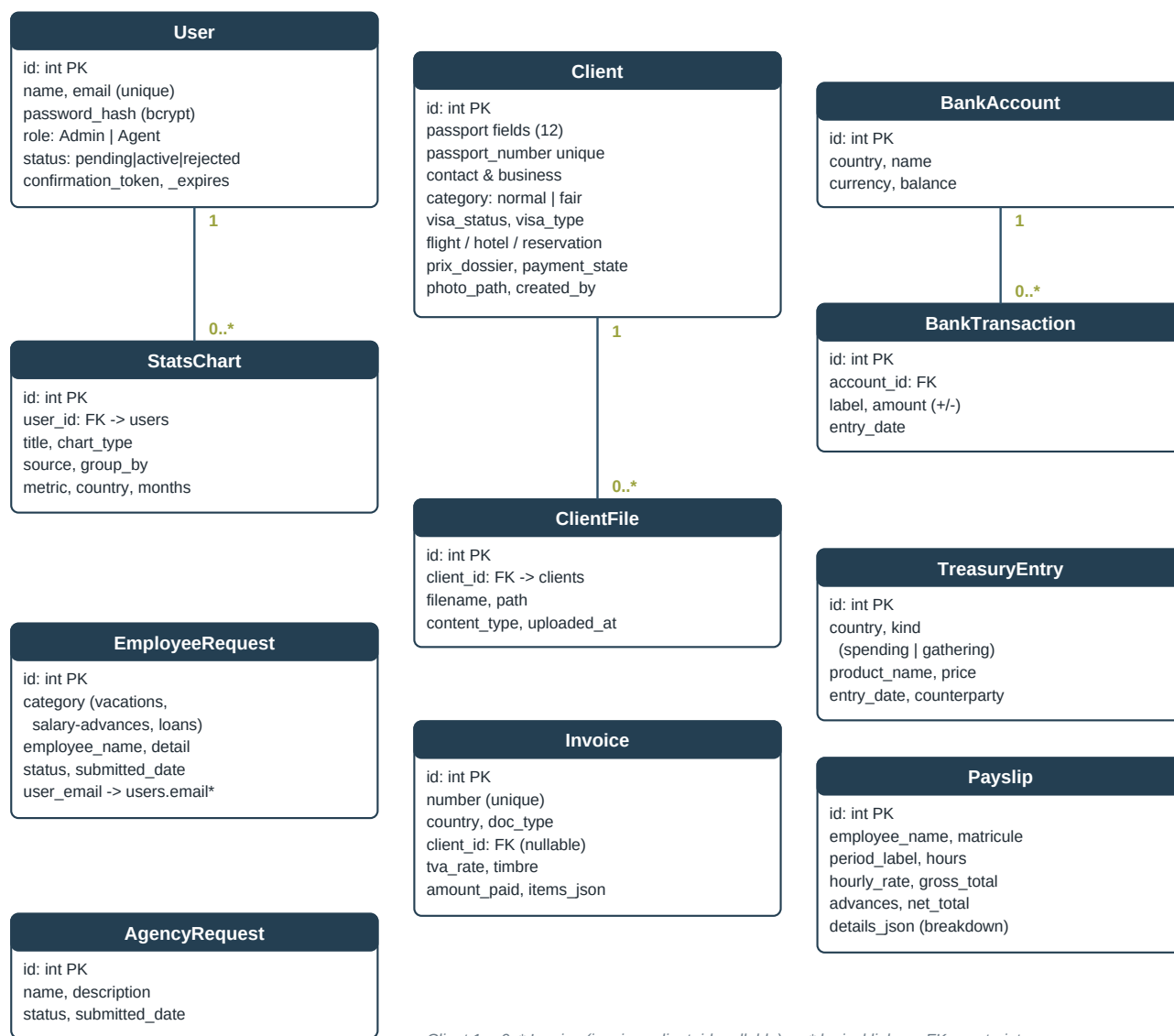


Figure 4 — UML class diagram of the persistent model (main attributes).

5.1 Tables

Table	Purpose and notable constraints
users	Accounts. email unique; password_hash (bcrypt); role Admin/Agent; status pending/active/rejected; one-time confirmation_token valid 72 h.
clients	The client/visa file. passport_number unique and indexed; ~40 columns grouped as passport, contact/business, category, visa, travel/reservation, pricing; photo_path, created_by.
client_files	Documents attached to a client (FK client_id, indexed).
invoices	Factures and reçus. number unique (e.g. FAC - TN - 2026 - 0007); line items stored as JSON in items_json; optional FK to clients.
treasury_entries	Cash ledger per country: spending or gathering, amount, date, who recorded it, counterparty.
bank_accounts / bank_transactions	Accounts per country with a running balance; each transaction (+deposit / -withdrawal) updates the balance in the same transaction.
agency_requests	Admin-to-staff requests with status.
employee_requests	Staff requests (vacations, salary advances, loans) with status; user_email records the author for privacy filtering.

Table	Purpose and notable constraints
payslips	Generated payslips: totals plus details_json, the full computed breakdown (summary lines, daily detail, parameters) so the PDF can be re-rendered identically.
stats_charts	Saved chart-builder definitions, FK to users (each user has their own charts).

5.2 Design decisions worth explaining

- **JSON columns for nested data** (invoice items, payslip breakdown): these are always read and written as a whole with their parent, never queried individually, so a child table would add joins without benefit.
- **Derived values computed in SQL**: dashboard counts, monthly series and alerts use COUNT, GROUP BY and to_char(created_at, 'YYYY-MM') instead of downloading every row to the client.
- **Migrations**: 12 Alembic versions document the schema's evolution (users, payslips, client travel fields, payslip breakdown...). Upgrading a server is alembic upgrade head.
- **Deleting a client** first deletes its client_files rows and commits, then deletes the client: without ORM relationships SQLAlchemy does not guarantee FK-safe ordering inside one flush.

6. Backend

6.1 Application structure

backend.py creates the FastAPI app, installs CORS and an authentication middleware, and declares all routes. Domain logic is kept in service modules (payroll, invoices, OCR, stats, assistant context, filters) so it can be tested and reused without HTTP. Each request gets its own database session through a dependency:

```
def get_session():
    with Session(engine) as session:
        yield session

@app.get("/clients/{client_id}")
def get_client(client_id: int, session: Session = Depends(get_session)):
    ...
```

Slow, blocking work (OCR, LLM calls, building the assistant context) runs in FastAPI's thread pool with run_in_threadpool, so one long AI request does not block other users.

6.2 Authentication and authorization

Passwords are hashed with bcrypt (salted, slow by design). **Sessions** are JWTs signed with HS256 using JWT_SECRET from .env; the payload carries sub (user id), email, role and exp. A normal session lasts 12 hours; "remember me" issues a 30-day token and the web client stores it in localStorage instead of sessionStorage; the iOS app stores it in the Keychain.

Authorization happens at two levels:

- **Global middleware** require_auth: every path except a public allow-list (/auth/signup, /auth/login, the e-mail confirm/reject links, API docs) needs a valid token, taken from the Authorization header or a ?token= query parameter (needed for and PDF links, which cannot send headers). Paths under /treasury, /banking, /invoices, /payroll, /agency-requests additionally require role == Admin.
- **Route dependencies**: sensitive endpoints declare Depends(require_admin) (user management, request approval, deletions) or Depends(get_current_user) when the result depends on who asks (an agent only lists their own employee requests; the assistant only sees data allowed for the role). This defence in depth means a missing prefix cannot silently expose an admin action.

Sign-up with approval (sequence below): the new account is created as *pending* with a random token; an e-mail with Approve / Reject links goes to ADMIN_EMAIL; the link activates the account. Login refuses pending or rejected accounts.

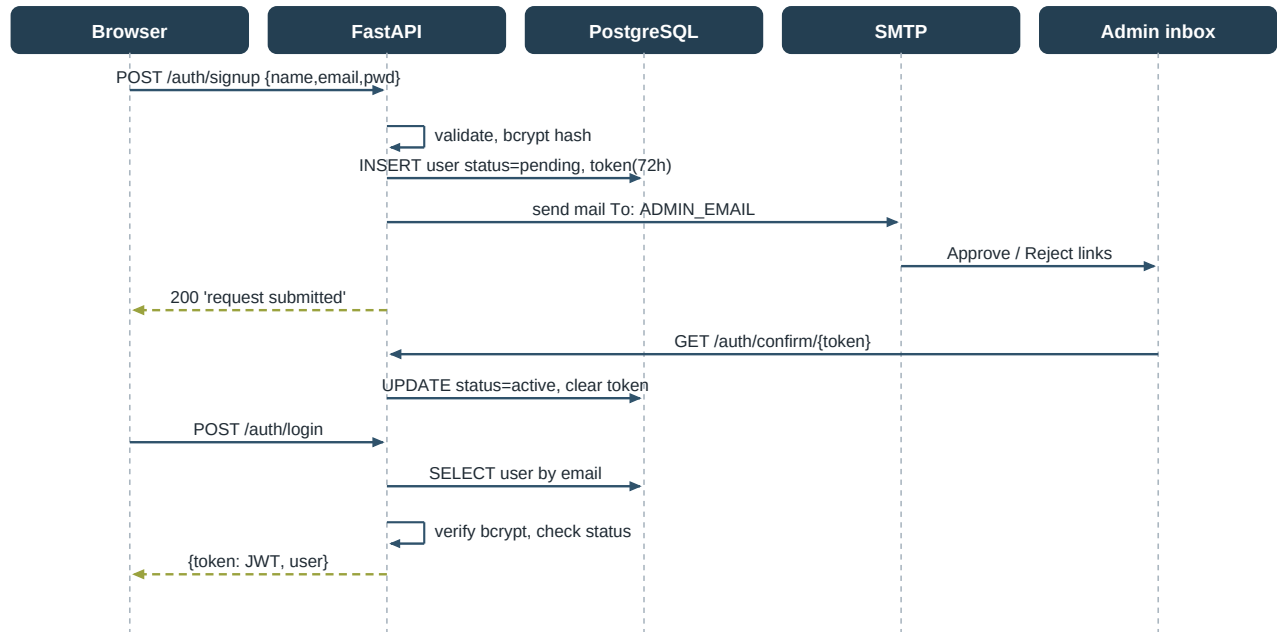


Figure 5 — Sequence: account request, e-mail approval and login.

6.3 REST API reference

54 endpoints, grouped by resource. JSON in and out; errors use HTTP status codes with {"detail": ...}. Interactive documentation is generated at /docs (OpenAPI).

Method & path	Purpose	Access
POST /auth/signup, /auth/login; GET /auth/me	Account request, sign-in (returns JWT), current user	public / auth
GET /auth/confirm/{t}, /auth/reject/{t}	E-mail approval links	public (token)
GET /auth/pending, /auth/users; POST /auth/users/{id}/approve reject role; DELETE /auth/users/{id}	User administration (cannot change or delete yourself)	Admin
GET /clients?q&tab&status&limit&offset	Search + paging; total in X-Total-Count header	auth
GET /clients/summary	Dashboard/Stats counts computed in SQL	auth
GET PUT DELETE /clients/{id}; POST /clients	Client CRUD (409 on duplicate passport)	auth
GET /clients/{id}/photo; GET POST /clients/{id}/files; GET DELETE .../files/{fid}	Photo and documents	auth
POST /extract?engine=auto ocr ai	Passport photo to fields + per-field confidence	auth
POST /chat	AI assistant (messages + optional image)	auth
GET POST /treasury; DELETE /treasury/{id}	Ledger with current month and history	Admin
/banking/accounts (+ /{id}/transactions)	Accounts and transactions (balance kept in sync)	Admin
GET POST /invoices; PUT DELETE /invoices/{id}; GET /invoices/{id}/pdf	Factures / reçus and PDF	Admin

Method & path	Purpose	Access
GET POST DELETE /agency-requests	Admin-to-staff requests	Admin
GET POST /employee-requests	Own requests (agent) or all (admin)	auth
PUT /employee-requests/{id}/status; DELETE ...	Approve / reject / remove	Admin
POST /payroll/parse-pointage	Time-clock file to employees + daily rows	Admin
GET POST /payroll/payslips; GET .../{id}/pdf; DELETE .../{id}	Payslips and Arabic PDF	Admin
POST /stats/query; GET POST /stats/charts; DELETE /stats/charts/{id}	Chart builder (role-filtered sources)	auth

6.4 Passport extraction pipeline

Typing passport data is the agency's most repetitive task, and an LLM alone is slow (tens of seconds on CPU) and can invent digits. The extractor therefore uses **deterministic OCR first and the AI only as a fallback**:

- **OCR** (RapidOCR, ~1 s): the image is normalised (resized to 1300-1600 px, rotations tried) and all text lines are read; Tesseract can re-read the bottom strip if installed.
- **Find the MRZ**: the two 44-character machine-readable lines (ICAO 9303, format TD3) are located by shape.
- **Repair** typical OCR slips: filler '<' read as K/C/S/E/L/I/X, and letter/digit confusions (O-0, I-1, S-5, B-8...) depending on whether a position must be a digit or a letter.
- **Verify with check digits**: passport number, birth date, expiry and the composite each carry a check digit; a field that passes is mathematically confirmed (*verified*).
- **Complete** printed-only fields: names cross-checked with the visual zone; issue date derived from the expiry minus the country's validity period (Tunisia 5 years, Libya 8) and confirmed against the printed date; place of birth and issuing authority read from the page.
- **Fallback**: if no MRZ is found, the image goes to the Qwen2.5-VL model with a strict JSON prompt, and every field is marked *review*. The response includes which engine answered and how long it took.

```
def check_digit(data: str) -> int:          # ICAO 9303
    total = 0
    for i, ch in enumerate(data):
        # digits keep their value, letters A=10 .. Z=35, filler '<' = 0
        v = int(ch) if ch.isdigit() else (ord(ch) - 55 if ch.isalpha() else 0)
        total += v * (7, 3, 1)[i % 3]
    return total % 10
```

6.5 AI assistant with live business context

A small local model knows nothing about the agency, so each question is answered with a **fresh snapshot of the database** appended to the system prompt (a lightweight form of retrieval-augmented generation).

`assistant_context.build_context(session, user, question)` collects: clients whose name or passport matches words in the question (max 5), counts by visa status, and, **only for admins**, treasury, bank balances, invoices and payroll; an agent only gets their own employee requests. The prompt instructs the model to quote exact figures and to say when the data does not contain the answer.

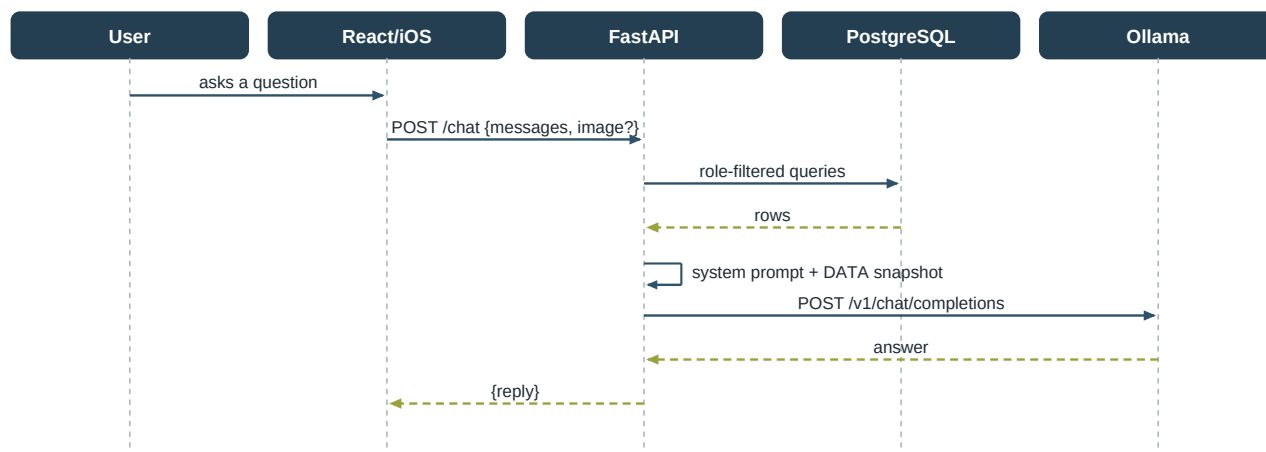


Figure 6 — Sequence: assistant question answered from live data.

6.6 Payroll engine (fiche de paie)

The agency's ZKTeco K40 time clock exports one row per employee per day (date, schedule, start/end of shift, punch-in/out, absent, weekend and holiday flags, planned and actual presence). `payroll.parse_pointage` detects the format by its first bytes (.xlsx is a ZIP; the device's .xls is a raw BIFF stream read with `xlrd`), finds columns by header prefix, normalises time cells to HH:MM, and groups rows by employee. `compute_payslip` then classifies each day:

Day type	Rule	Counted as
Absent	flagged absent, or no punch at all	absence (no pay)
Public holiday	Holiday flag	actual minutes x rate x (1 + 100%)
Weekend	WeekEnd flag or schedule containing "weekend"	actual minutes x rate x (1 + 50%)
Normal	otherwise	planned minutes x rate; overtime = max(actual - planned, 0) x rate x (1 + 25%)

Planned presence uses the device value when present, otherwise *end of shift* - *max(arrival, start)* minus the lunch break (default 60 min, start 08:00, end 16:30); actual presence uses the device value, otherwise planned + minutes worked after the end of shift. **Net = base + overtime + weekend + holiday - advances**. All majorations and the break are parameters.

The rules were ported line-for-line from the agency's reference tool (`paie_app.html`) and **validated by running the original JavaScript and the Python port on the same real exports**: every hour total, amount and daily classification matched exactly. The result, including the daily detail, is stored in `details_json` and rendered as an A4 Arabic payslip with XeLaTeX.

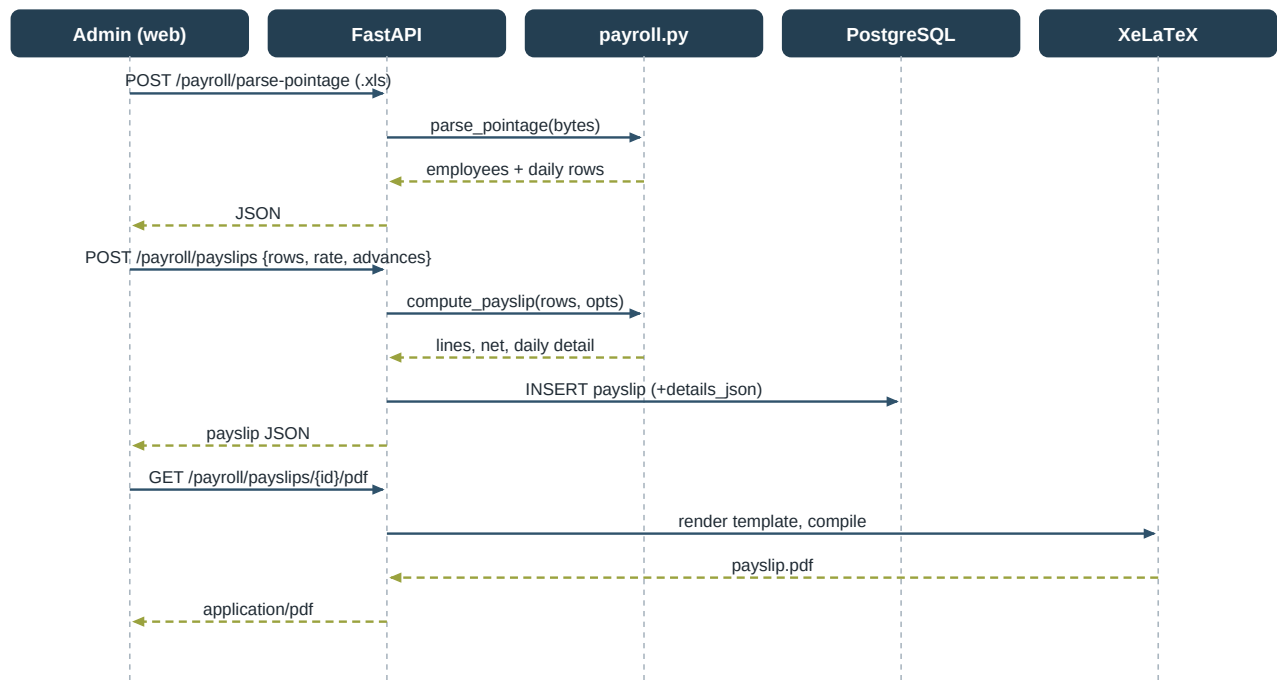


Figure 7 — Sequence: payslip from time-clock file to PDF.

6.7 Other business rules

- **Invoice numbering:** {FAC | REC} - {TN | LY} - {year} - {n:04d}, n = documents of that type, country and year + 1. Factures compute HT, VAT (19% default) and a fixed stamp duty (1 TND).
- **Client alerts** (single source of truth in `client_filters.py`, shown on web and iOS): the passport is ready but the client has not been notified, or the flight is within 10 days and the visa is not yet treated.
- **Client search:** every word typed must match the name, passport, phone or e-mail (case-insensitive ILIKE); results are paged with `limit/offset`.
- **Chart builder** (`stats.py`): a query is {source, group_by, metric, country, months}; each source declares its allowed dimensions and metrics (a whitelist, so user input never becomes raw SQL); money sources are refused for agents; time dimensions sort chronologically; at most 25 groups.
- **Banking:** adding or deleting a transaction adjusts the account balance in the same database transaction.

7. Web frontend (React)

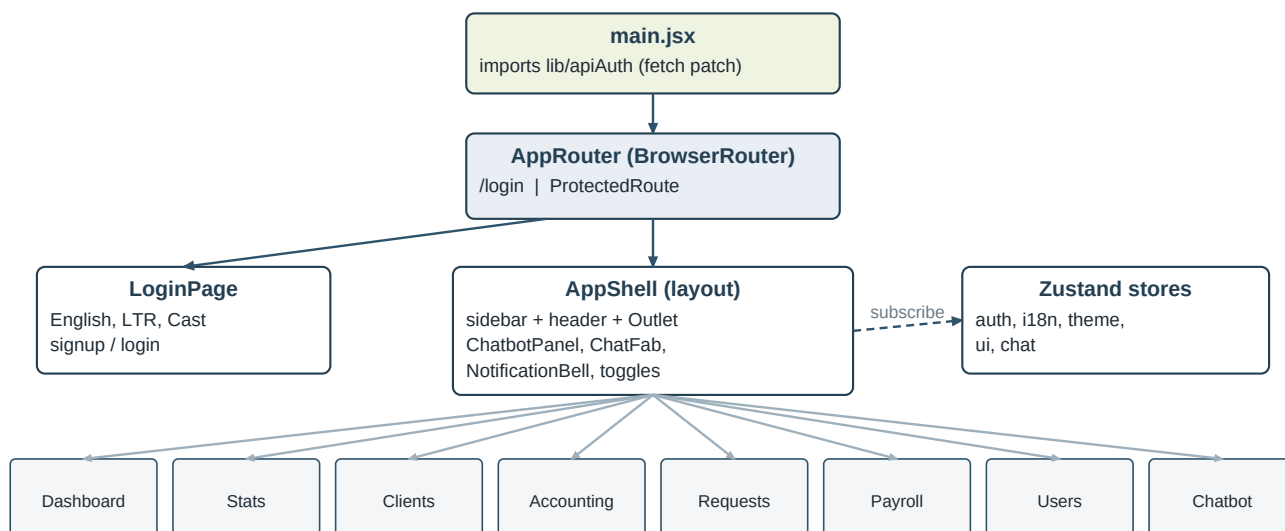


Figure 8 — Component tree and state.

7.1 Structure

- **app/**: router (public `/login`; everything else behind `ProtectedRoute` and the `AppShell` layout), route transitions.
- **features/**: one folder per business area (dashboard, clients, accounting, requests, payroll, stats, chatbot, admin, auth); each page fetches its own data.
- **components/**: shared UI (buttons, inputs, cards, icons, toasts, notification bell, global search, toggles).
- **store/**: Zustand stores; **lib/**: fetch patch, API base, GSAP/Lenis setup, paging hook; **i18n/**: 1,200-line EN/AR dictionary; **styles/**: tokens, base, layout, components, pages.

7.2 State management

Store	Holds	Notes
auth	user, token, login/signup/logout	Persists to localStorage (remember me) or sessionStorage; <code>withToken(url)</code> appends the token to image/PDF URLs.
i18n	lang, isRtl, t(key)	t is recreated on every language change so subscribed components re-render; sets <code><html dir=rtl></code> .
theme	light / dark	Sets data-theme; respects the OS preference.
ui / chat	sidebar, chatbot panel, messages	Sidebar starts closed below 900 px.

7.3 Internationalisation, theming and responsiveness

- **Arabic RTL**: layout uses CSS logical properties (`margin-inline-start`, `inset-inline-end`, `text-align: start`) so the whole UI mirrors automatically when `dir="rtl"`; the login page is deliberately English-only.
- **Design tokens**: all colours are CSS variables in `variables.css`; dark mode overrides them under `:root[data-theme="dark"]` with a Material-style palette.
- **Responsive**: below 900 px the sidebar becomes an overlay drawer; padding and header controls shrink at 640/480 px; `env(safe-area-inset-*)` keeps content clear of the iPhone notch.

- **Large lists:** the clients table pages through the API (usePagedList reads X-Total-Count) instead of loading every client.

8. Native iOS application (SwiftUI)

The iPhone app gives the owner the same data on the move. It is a native SwiftUI app (iOS 17+, ~6,100 lines, no external dependencies) that talks to the same REST API with the same JWT, so every rule (roles, alerts, payroll, chart queries) is enforced once on the server.

Views (SwiftUI)

LoginView, MainTabView (5 tabs), Dashboard, Clients, Accounting, Requests, Payroll, Stats, Chat, Users, Search, Settings

State (ObservableObject, @MainActor)

AuthStore (session, Keychain), AppSettings (server URL, language, theme), AlertsStore, ChatStore

Networking

API singleton: URLSession + async/await, Bearer token, snake_case <-> camelCase Codable, multipart upload, PDF download

Models & utilities

Codable structs mirroring the backend JSON (Models.swift), L10n EN/AR, Theme, Toast, Media/PDF preview

Same REST API and JWT as the website -> one backend, two clients

Figure 9 — Layers of the iOS app.

- **App flow:** TCAERPApp injects AppSettings and AuthStore as environment objects; RootView shows LoginView without a token, otherwise MainTabView with five tabs (Dashboard, Clients, Accounting for admins, Requests, More).
- **Networking:** an @MainActor API singleton built on URLSession and async/await; JSON keys convert between snake_case and camelCase automatically; multipart uploads for passports and time-clock files; PDFs are downloaded and previewed natively.
- **Security:** with "remember me" the token is stored in the iOS Keychain; logging out clears it.
- **Features:** dashboard, clients (list, detail, form with passport extraction), accounting, banking, requests, payroll (same ZKTeco flow as the web), stats and custom charts, AI chat, users, global search, settings (server address with a connection test, language, theme).
- **Localisation:** EN/AR strings with the layout direction flipped for Arabic; the login stays English, as on the web.

9. Security review

Control	Implementation
Password storage	bcrypt with per-password salt; never logged or returned.
Session tokens	HS256 JWT with expiry (12 h / 30 days); secret only in .env; 401 triggers client logout.
Account creation	Pending until approved by the admin through a one-time, 72-hour e-mail token.
Authorization	Middleware for every request + route-level require_admin; data filtered by role (requests, assistant, chart sources).
Injection	ORM-built queries; chart builder uses a whitelist of dimensions; LaTeX inputs escaped; React escapes output (no innerHTML).
Uploads	Client photos validated and re-encoded to JPEG with Pillow; document file names sanitised and prefixed with a UUID; files served only through authenticated endpoints with an explicit content type.
Secrets	.env is git-ignored; .env.example documents the variables without values.

9.1 Known limitations and planned hardening

- **Before internet exposure:** restrict CORS (currently *), disable reload=True and the public /docs, add login rate limiting, serve only over HTTPS.
- **Invoice number race:** count-then-insert can collide under concurrent creation; fix with a database sequence or a retry on the unique constraint.
- **Token in query string** for images/PDFs can appear in server logs; short-lived signed URLs would be cleaner.
- **Tests:** unit tests cover the MRZ parser; the payroll port was validated against the reference implementation; API integration tests are the next step.

10. Deployment and operations

Target: the agency's own server, reachable from anywhere so the owner can use the iPhone app outside the office.

Item	Plan
Packaging	Docker Compose with four services: postgres, backend (with TeX Live and OCR models), nginx (React build + reverse proxy) and ollama; one command to start or update.
Public access	A domain name with a Cloudflare Tunnel: HTTPS, no fixed IP or router changes, server IP hidden.
Hardware	8-core CPU, 32 GB RAM, NVIDIA RTX 4060 Ti 16 GB (AI answers in ~2-5 s, several parallel requests), 500 GB SSD, Ubuntu Server 24.04 LTS, UPS; a 4G backup router for internet outages.
Configuration	Per-install .env: DATABASE_URL, JWT_SECRET, ADMIN_EMAIL, SMTP_*, APP_BASE_URL, PUBLIC_API_BASE_URL.
First run	alembic upgrade head; python -m backend.seed_admin; ollama pull qwen2.5vl:3b-8k.
Backups	Nightly pg_dump plus the uploads/ folder, copied off the server.
iOS release	Apple Developer Program; App Store (unlisted distribution) or TestFlight; needs an in-app account deletion option and a privacy-policy URL for review.

11. Challenges solved

- **Arabic in PDFs:** XeTeX's bidi reversed Latin words and scrambled colours inside right-to-left runs; solved by keeping Latin values and colour changes outside Arabic runs and reversing table columns in the template.
- **Legacy Excel:** the time clock writes raw BIFF .xls files without an OLE container; detected by magic bytes and read with xlrd.
- **Reliable passport data:** replacing "ask the LLM" with OCR plus check digits turned a 30-60 s, sometimes wrong, step into a ~2 s, verifiable one.
- **React re-render bug:** language changes did not refresh screens because the translate function kept the same reference; fixed by recreating it on each change.
- **Foreign-key ordering:** client deletion failed until dependent files were deleted and committed first.
- **Native app networking:** localhost and the Vite proxy do not exist on a phone; added a configurable server URL, App Transport Security and Local Network permission entries.

12. Interview preparation: likely questions

Why FastAPI rather than Django or Express?

The API is the product (two clients consume it), so I wanted typed request/response models, automatic validation and OpenAPI docs with little code. FastAPI gives that, plus async support for the slow AI and OCR calls. Django's admin and templates were not needed; Python was needed anyway for OCR, Excel and LaTeX tooling.

How does authentication work end to end?

Sign-up stores a bcrypt hash with status pending and e-mails the admin an approval link containing a random token valid 72 h. Login verifies the hash and status and returns a JWT (HS256) with the user id, role and expiry. Clients send it as a Bearer header; a middleware validates it on every request and blocks admin-only path prefixes, and sensitive routes add a `require_admin` dependency.

JWT vs server sessions — why JWT?

Stateless verification works the same for the web and the iOS app, with no session store. The trade-off is that a token cannot be revoked before it expires, so lifetimes are bounded (12 h, or 30 days with remember me) and rotating `JWT_SECRET` invalidates all tokens at once.

How do you stop an Agent from seeing financial data?

Three layers: the UI hides admin pages; the middleware rejects `/treasury`, `/banking`, `/invoices`, `/payroll` and `/agency-requests` for non-admins; and data built per user (assistant context, chart sources, employee requests) is filtered by role on the server. The UI is never trusted.

Walk me through the database model.

Twelve tables: users; clients with `client_files`; invoices (optional FK to clients); treasury entries; bank accounts with transactions; agency and employee requests; payslips; saved stats charts (FK to users). Nested data that is always read with its parent, like invoice items or a payslip breakdown, is stored as JSON.

How did you make passport extraction fast and trustworthy?

OCR finds the two machine-readable lines, I repair known OCR confusions and validate each field with the ICAO 9303 check digits (weights 7-3-1, modulo 10). A field that passes is certain. Only when no MRZ is found do we fall back to the vision LLM, and those fields are marked for review.

How does the chatbot know about your data if the model is small and local?

Each question is sent with a compact snapshot of live data in the system prompt: matching clients, counts and, for admins, finances. It is a simple retrieval step; the model only phrases the answer and is told to say when the data does not contain it.

How did you verify the payroll calculation?

I ported the agency's existing JavaScript tool rule by rule, then ran the original JS and my Python on the same real time-clock exports with different parameters and compared every line, total and day classification: identical.

How is the app bilingual with right-to-left support?

A dictionary per language and a `t(key)` function in a Zustand store; switching to Arabic sets `dir=rtl` on the html element, and the CSS uses logical properties so layouts mirror automatically. iOS does the same with a layout direction environment value.

How do the web and iOS apps stay consistent?

All rules live in the backend: alert rules, chart computations, payroll, invoice totals. The clients only render. Shared JSON contracts are mirrored as Codable structs in Swift.

What would break first under more load, and how would you scale?

The local LLM (GPU-bound). Ollama queues requests; a bigger GPU or several workers scale it. The API is stateless, so it can run multiple uvicorn workers behind Nginx; PostgreSQL easily handles this size; heavy list endpoints already page in SQL.

What are the weaknesses you would fix next?

Lock down CORS and docs, add rate limiting before public exposure, replace the invoice counter with a sequence, use short-lived signed URLs for files, add API integration tests, and package everything with Docker Compose.

How do you manage schema changes?

Alembic: each change is a migration script with upgrade and downgrade; deployment runs alembic upgrade head. Data migrations, such as backfilling a new column, go in the same script.

Why generate PDFs with LaTeX instead of HTML-to-PDF?

Precise, print-quality layout and good Arabic shaping with XeLaTeX and the Amiri font, without a headless browser on the server. Templates are plain .tex files with placeholders, easy to adjust.

Appendix A — Configuration variables

Variable	Meaning
DATABASE_URL	PostgreSQL connection string (postgresql+psycopg://user:pass@host:5432/db)
JWT_SECRET	Long random secret used to sign tokens (required; the backend refuses to start without it)
JWT_EXPIRES_HOURS / JWT_REMEMBER_DAYS	Token lifetimes (12 h / 30 days)
ADMIN_EMAIL	Address that receives account-approval e-mails and owns the seeded admin account
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM	Mail server (Gmail with an app password)
APP_BASE_URL, PUBLIC_API_BASE_URL	Public addresses used to build links in e-mails

Appendix B — Glossary

Term	Definition
MRZ	Machine-Readable Zone: the two coded lines at the bottom of a passport (ICAO Doc 9303).
JWT	JSON Web Token: signed, self-contained token proving identity and role until it expires.
ORM	Object-Relational Mapper: Python classes mapped to database tables (SQLModel/SQLAlchemy).
Migration	Versioned script that changes the database schema (Alembic).
RTL	Right-to-left text direction, used for Arabic.
Fiche de paie	Payslip (French); the Arabic document title is 'payslip card'.
Facture / Reçu	Invoice with VAT and stamp duty / payment receipt.
TND / LYD	Tunisian dinar / Libyan dinar (3 decimals).
Ollama	Local server that runs open LLMs and exposes an OpenAI-compatible API.
Reverse proxy	Server (Nginx) in front of the app that handles HTTPS and routes requests.